QUALITY ASSURANCE · PRACTICE DOCUMENT

Test Plan

Restful Booker API

Document purpose
Practice example

Plan status
Draft — not approved for project execution

Prepared
November 7, 2026

Reference
Workspace test plan creator skill

Practice-only notice: This plan is based on the Restful Booker example and template content in the workspace skill. It is not based on a separately supplied PRD. Confirm requirements, environment readiness, dates, roles, and acceptance thresholds with project stakeholders before execution.

Contents

1. Objective
2. Scope
3. Inclusions
4. Test Environments
5. Defect Reporting Procedure
6. Test Strategy
7. Test Schedule
8. Test Deliverables
9. Entry and Exit Criteria
10. Test Execution
11. Test Closure
12. Tools
13. Risks and Mitigations
14. Approvals

1. Objective

The objective is to assess the quality, functionality, and reliability of the Restful Booker API at https://restful-booker.herokuapp.com, focusing on booking create, read, update, and delete (CRUD) flows, input validation, authentication, and API responses.

Testing will also consider response behavior, error handling, data consistency, and the API documentation. Experiment impact metrics and guardrail metrics are not defined in the supplied information and must be agreed before any experiment-related testing.

2. Scope

In scope

- Functional and CRUD testing of documented booking operations.
- Input and response data validation, including invalid, missing, and boundary values.
- Authentication and authorization checks for protected operations.
- Error handling, integration/data consistency, regression, and exploratory testing.
- Performance and load behavior, concurrency, rate limiting, and security checks at a test-plan level.
- API documentation review and CI/CD behavior where the relevant pipeline is available.

Out of scope / not specified

- Unspecified third-party services, product features, service-level objectives, and compliance requirements.
- Production data changes or destructive testing outside an authorized test environment.
- Specific performance thresholds, load volumes, supported API versions, and detailed test data; these require stakeholder agreement.

3. Inclusions

Planned coverage areas

Area	Coverage
Create (POST)	Create bookings with valid data; check invalid or missing fields and verify the created booking can be retrieved.
Read (GET)	Retrieve booking information using documented criteria; verify returned data and handling of invalid or nonexistent identifiers.
Update (PUT)	Update existing bookings with valid data; check invalid updates and confirm persisted values.
Delete (DELETE)	Delete a booking using a valid identifier; verify the response and subsequent state.
Data and boundaries	Check required fields, types, invalid characters, and minimum/maximum or near-boundary values where rules are documented.
Authentication and access	Check CRUD behavior for authenticated and unauthenticated users and verify authorization restrictions described by the API.
Reliability and security	Check malformed requests, sensitive-data exposure, secure transport, concurrency, rate limiting, and graceful error behavior.
Quality attributes	Assess response time, throughput, scalability, compatibility, documentation accuracy, and regression behavior. Thresholds require agreement.

4. Test Environments

Environments and platforms recorded in the supplied template

Environment	URL	Notes
QA	API documentation	Environment endpoint details are not separately specified.
Pre-Prod	API documentation	The supplied template lists the same URL for QA and Pre-Prod; confirm whether these are distinct environments.

Platforms listed: Windows 10 with Chrome, Firefox, and Edge; macOS with Safari; Android with Chrome; iPhone with Safari.

Confirm actual OS/browser versions, device coverage, network conditions, test credentials, permissions, and environment-specific base URLs before execution.

5. Defect Reporting Procedure

1. Record each observed deviation from documented requirements or expected behavior in JIRA.
2. Include a concise title, environment, preconditions, reproducible steps, expected and actual results, severity/priority, and relevant evidence or logs.
3. Triage defects with the appropriate project stakeholders; track ownership, status, and retest outcome in JIRA.
4. Share a daily end-of-day defect status with development management, as described in the supplied template.

Contacts/ownership stated in the supplied template

Area	Named contact
Frontend	Devesh
Backend	Sonal
DevOps	Prajeeth
POC	Not specified

Confirm that these contacts and the defect triage workflow are current before use.

6. Test Strategy

1. **Design:** Derive scenarios and test cases from confirmed requirements and API documentation. Apply equivalence partitioning, boundary value analysis, decision tables, state transition testing, use-case testing, and error guessing.
2. **Prioritize:** Prioritize cases by business and technical risk once impact and severity criteria are agreed.
3. **Smoke and sanity:** Run smoke checks on a candidate build. If critical smoke checks fail, report the failure and request a stable build before in-depth execution.
4. **Execution:** Run functional, validation, integration, authentication, regression, compatibility, and exploratory checks across confirmed supported environments.
5. **Quality attributes:** Conduct performance, load, concurrency, and rate-limit checks only with approved limits, a suitable environment, and agreed thresholds.
6. **Defects and retesting:** Log defects in JIRA, retest fixes, and repeat relevant regression coverage until agreed exit criteria are met.

Apply context-driven, shift-left, exploratory, and end-to-end testing practices. Multiple testers may execute across supported environments in parallel when access and test data permit.

7. Test Schedule

The supplied template indicates a two-sprint testing window. Calendar dates and task durations were not provided.

Task	Timing
Test plan preparation and approval	Date to be confirmed
Test scenario and test case creation	Date/duration to be confirmed
Test execution, defect retesting, and regression	Within the planned two sprints; detailed dates to be confirmed
Summary report submission	Date to be confirmed

8. Test Deliverables

- Approved test plan.
- Test scenarios and prioritized test cases.
- Execution results and test case reports.
- JIRA defect records and defect status reports.

- Test summary report and closure notes.
- Supporting evidence, such as screenshots or logs, where appropriate.

9. Entry and Exit Criteria

Phase	Entry criteria	Exit criteria
Requirement analysis	Testing team receives the requirements or project details.	Requirements are explored and understood; open questions are clarified or recorded.
Test execution	Test scenarios/cases are signed off by the client; application is available for testing; access and test data are ready.	Planned execution is recorded; test case and defect reports are available; unresolved risks are communicated.
Test closure	Test case and defect reports are ready.	Test summary report is prepared and shared; closure status and outstanding items are documented.

Quantitative exit thresholds (for example, pass rate, open-defect limits, or response-time targets) were not supplied and must be agreed before execution.

10. Test Execution

- Verify the target environment, build, credentials, permissions, and test data before starting.
- Run and record smoke tests; communicate blockers before proceeding to detailed testing.
- Execute prioritized cases and exploratory checks; capture actual results and evidence.
- Log failures in JIRA, retest fixes, and run related regression checks.
- Summarize progress, defects, blockers, and risks to stakeholders on the agreed cadence.

11. Test Closure

At the end of the test cycle, consolidate execution results, defect status, outstanding risks, coverage, and any deviations from the plan in a test summary report. Obtain stakeholder review and record the agreed closure or follow-up actions.

12. Tools

- JIRA — defect tracking.
- Mind map tool — test analysis and scenario organization.
- Screenshot capture tool — evidence collection.
- Word and Excel documents — planning, cases, and reports.

13. Risks and Mitigations

Risk	Mitigation stated in the template
Resource unavailable	Plan a backup resource.
Build URL unavailable	Use the time for other testing tasks while access is restored.
Insufficient testing time	Scale resources dynamically based on client needs.

Environment access, unclear requirements, unspecified test thresholds, and shared test data may also affect execution; owners and mitigations should be agreed during project planning.

14. Approvals

The supplied process calls for client approval of the test plan, test scenarios, test cases, and reports before proceeding to subsequent steps.

Document	Approver	Status / date
Test plan	Client/stakeholder — name not supplied	Pending
Test scenarios and cases	Client/stakeholder — name not supplied	Pending
Test reports	Client/stakeholder — name not supplied	Pending

Practice document generated from the workspace test plan creator skill. Confirm all project-specific details before treating this plan as an execution baseline.